SERVICIO NACIONAL DE APRENDIZAJE — SENA

República de Colombia



MANUAL TÉCNICO

Frontend

SPA React, rutas, estado, integración, pruebas y normas ISO

Componente: Frontend web

Proyecto: ProyecTwin-MVP — plataforma de gestión y análisis de similitud de propuestas formativas

Versión documental: 4.2 — 4 de noviembre de 2026

Commit de referencia: 1ebab35bf580c9e7ce912fb6aa919e182ebaf212

Repositorio: https://github.com/YDanB-312/ProyecTwin-MVP (rama master)

Estado: Documento técnico para revisión; no declara certificación ISO ni conformidad formal

Documento elaborado a partir de la auditoría técnica del repositorio y de la verificación de versiones, rutas, componentes, estado, pruebas y configuración. Las afirmaciones se sustentan en archivos concretos; lo que no pudo comprobarse se marca como PENDIENTE DE VERIFICACIÓN.

Ficha de identificación del documento

Tabla 1. Ficha del documento

Campo	Valor
Nombre del documento	Manual técnico del frontend web
Identificador documental	PT-MT-FE-03
Versión	4.2
Fecha	4 de noviembre de 2026
Commit de referencia	1ebab35bf580c9e7ce912fb6aa919e182ebaf212
Repositorio	https://github.com/YDanB-312/ProyecTwin-MVP — rama master
Alcance	SPA React 19 + Vite 8, rutas por rol, componentes, estado, consumo de la API, seguridad, pruebas y despliegue.
Responsable	Auditoría técnica y generación documental asistida
Estado	Completo para revisión del equipo. No sustituye al código ni a la ejecución del sistema. No es una auditoría de certificación.

Fuente: Elaboración propia a partir de la auditoría del repositorio.

Control de versiones del documento

Se conserva el historial anterior. La versión 3.0 corrigió los desfases entre los manuales v2.0 (28-oct-2026) y el código del commit de referencia. La versión 4.1 se identificó en la portada del ejemplar auditado, pero no tenía una fila propia en este historial; la fila se reconstruye como derivación editorial. La versión 4.2 documenta esta auditoría.

Tabla 2. Historial de versiones

Versión	Fecha	Descripción del cambio	Responsable
2.0	28/10/2026	Versión anterior de los manuales v2; auditoría estática y pruebas parciales.	Equipo ProyecTwin
3.0	03/11/2026	Reescritura integral contra el commit 1ebab35; corrección de versiones, rutas, componentes, pruebas, seguridad y despliegue; integración ISO 25010/12207/29148/27001/9001/25023/29119.	Auditoría técnica y generación documental asistida
3.1	03/11/2026	Auditoría de coherencia: estado del APK, referencias normativas complementarias, matrices de hallazgos y control de cambios.	Auditoría técnica y generación documental asistida
4.1	03/11/2026	Edición independiente derivada de la separación documental; portada 4.1. No se localizó una fila de historial propia en el ejemplar auditado.	No consta en el ejemplar auditado
4.2	04/11/2026	Auditoría y corrección integral: conteos de páginas, rutas y pruebas; variables de entorno; anexos únicos; alcance frontend; referencias ISO contrastadas; revisión visual de DOCX/PDF.	Auditoría técnica y documental

Fuente: Elaboración propia; la fila 4.1 se reconstruye a partir de la portada del ejemplar y no inventa aprobaciones.

Tabla de contenido

El índice es un campo automático de Word construido con los estilos Título 1, 2 y 3. Si aparece vacío, actualícelo con Referencias → Actualizar tabla (o F9). En el PDF se conservan los marcadores y enlaces.

1. Introducción 1

1.1 Actores y experiencias 1

2. Objetivo general 1
3. Objetivos específicos 1
4. Alcance 2
5. Descripción del frontend 2
5.1 Funcionalidades implementadas 3
6. Requisitos de hardware y software 4
7. Tecnologías y versiones exactas 4
7.1 Dependencias principales 4
8. Arquitectura 5
8.1 Decisión de mismo origen 6
9. Estructura de carpetas 6
10. Componentes 6
11. Páginas y módulos 7
12. Sistema de navegación 7
13. Gestión del estado 8
14. Formularios y validaciones 8
15. Servicios 9
16. Integración con backend 9
17. Variables de entorno 10
18. Autenticación 11
19. Manejo de errores 11
20. Instalación 11
21. Configuración 12
22. Ejecución local 12
23. Compilación 13
24. Pruebas 13

25. Seguridad 14
26. Calidad y normas ISO — Frontend 14
26.1 Normas y ediciones consideradas 14
26.2 ISO/IEC 25010 — evaluación por característica 15
26.3 ISO/IEC/IEEE 29148 — requisitos y trazabilidad 15
26.4 ISO/IEC/IEEE 12207 — procesos del ciclo de vida 16
26.5 ISO/IEC 27001 — controles y buenas prácticas de seguridad 16
26.6 ISO 9001 — gestión documental y calidad 16
26.7 ISO/IEC 25023 — métricas de calidad del producto 17
26.8 ISO/IEC/IEEE 29119 — pruebas de software 17
26.9 Matriz de trazabilidad y cumplimiento (resumen) 18
26.10 Normas complementarias 19
26.11 Reglas de uso normativo 19
27. Despliegue 19
28. Solución de problemas 20
29. Mantenimiento 20
30. Conclusiones 20
31. Anexos 21
31.1 Inventario completo de rutas 21
31.2 Scripts 21
32. Glosario 22
33. Referencias 22
33.1 Referencias normativas 22
33.2 Referencias técnicas del frontend 23
33.3 Archivos clave del frontend 23

Listas de figuras y tablas

Las listas se generan a partir de los campos de numeración de las leyendas. Actualice los campos si aparecen vacías.

Índice de figuras

Figura 1 . *Portal público de ProyecTwin (captura de referencia)* 3

Figura 2 . *Arquitectura lógica del sistema y posición de la SPA* 5

Figura 3 . *Ciclo de una petición autenticada* 10

Índice de tablas

Tabla 1 . *Ficha del documento* i

Tabla 2 . *Historial de versiones* i

Tabla 3 . *Actores del frontend* 1

Tabla 4 . *Funcionalidades del frontend* 3

Tabla 5 . *Requisitos para ejecutar el frontend* 4

Tabla 6 . *Tecnologías y versiones del frontend* 4

Tabla 7 . *Dependencias del frontend* 4

Tabla 8 . *Capas de la SPA* 5

Tabla 9 . *Componentes principales* 6

Tabla 10 . *Rutas representativas por rol* 7

Tabla 11 . *Control de acceso por rol* 8

Tabla 12 . *Estado y persistencia* 8

Tabla 13 . *Validaciones del cliente* 9

Tabla 14 . *Servicios por recurso* 9

Tabla 15 . *Flujo de datos* 10

Tabla 16 . *Variables del frontend* 10

Tabla 17 . *Flujo de autenticación web* 11

Tabla 18 . *Errores del cliente* 11

Tabla 19 . *Archivos de configuración* 12

Tabla 20 . *Evidencia de pruebas del frontend* 13

Tabla 21 . *Controles de seguridad del frontend* 14

Tabla 22 . *Evaluación ISO/IEC 25010:2023 del frontend web* 15

Tabla 23 . *Catálogo de requisitos verificables del frontend web* 15

Tabla 24 . *Procesos y actividades evidenciados* 16

Tabla 25 . *Prácticas de seguridad observadas* 16

Tabla 26 . *Prácticas de gestión de la calidad* 16

Tabla 27 . *Métricas ISO/IEC 25023:2016* 17

Tabla 28 . *Elementos de prueba ISO/IEC/IEEE 29119* 17

Tabla 29 . *Matriz de trazabilidad y cumplimiento — frontend web* 18

Tabla 30 . *Normas complementarias y aplicabilidad* 19

Tabla 31 . *Consideraciones de despliegue* 19

Tabla 32 . *Diagnóstico del frontend* 20

Tabla 33 . *Rutas declaradas en App.jsx* 21

Tabla 34 . *Scripts de package.json* 21

Tabla 35 . *Glosario técnico* 22

Tabla 36 . *Normas y ediciones de referencia* 22

Tabla 37 . *Documentación técnica de referencia* 23

Tabla 38 . *Mapa de archivos de referencia* 23

1. Introducción

El frontend de ProyecTwin-MVP es una SPA React que consume la API Laravel en /v1. Ofrece tres experiencias diferenciadas —aprendiz, instructor y administrador— y un portal público con información, demo de similitud y acceso. No contiene datos mock de la aplicación: todas las operaciones se resuelven contra la API real.

Este manual describe la SPA que existe en el repositorio, no una maqueta. Se apoya en App.jsx, src/pages, src/components, src/lib, los contextos, las pruebas y las capturas disponibles. Cuando una función no se pudo ejecutar en el entorno de auditoría se marca como no verificable o pendiente; no se inventan resultados.

Problema que resuelve. La SPA hace accesible el ciclo de propuestas y similitud a los tres roles: el aprendiz registra y envía propuestas, el instructor revisa y decide, y el administrador supervisa usuarios, catálogos, fichas, similitudes, soporte y bitácora. La interfaz publica el manual de usuario y muestra un botón de descarga Android solo si el backend anuncia una URL HTTPS; en la configuración actual el APK no se publica y el botón no aparece.

1.1 Actores y experiencias

Tabla 3. Actores del frontend

Actor	Experiencia	Rutas principales
Visitante	Portal público, demo, login, recuperación y activación.	/, /login, /recuperar-contrasena, /activar-cuenta
Aprendiz	Dashboard, propuestas, ficha, similitudes, alertas, perfil y soporte.	/aprendiz/*
Instructor	Dashboard, revisión, fichas, similitudes, alertas y perfil.	/instructor/*
Administrador	Usuarios, catálogos, fichas, propuestas, similitudes, bitácora, soporte y motor.	/admin/*
Sistema	Redirecciones heredadas y 404.	Rutas de compatibilidad y *

Fuente: frontend/src/App.jsx; frontend/src/constants/routes.js.

2. Objetivo general

Documentar de forma verificable la arquitectura, las rutas, los componentes, el estado, los servicios, la seguridad, las pruebas y la operación de la SPA de ProyecTwin-MVP para que un desarrollador nuevo pueda instalarla, comprenderla, mantenerla y desplegarla sin depender de explicaciones externas.

3. Objetivos específicos

- Identificar las versiones exactas de React, Vite, React Router, Vitest, Playwright, ESLint y Node.
- Describir la arquitectura de la SPA, la organización de carpetas y el flujo de una petición.

- Inventariar las 55 rutas declaradas y los 48 componentes de página, con su control de acceso.
- Explicar el estado de sesión, el tema, el motor, la caché de GET y el manejo de CSRF.
- Documentar formularios, validaciones, navegación, servicios y errores.
- Registrar la estrategia y evidencia de pruebas sin afirmar ejecuciones no realizadas en este commit.
- Aplicar ISO/IEC 25010:2023, 12207:2026, 29148:2018, 27001:2022, 9001:2026, 25023:2016 y 29119 como marcos de análisis, sin declarar certificación.

4. Alcance

Incluido: SPA React, portal público, autenticación, rutas por rol, componentes, layouts, contextos, servicios de API, formularios, validaciones, estilos, pruebas unitarias, E2E, build, variables de entorno y despliegue same-origin.

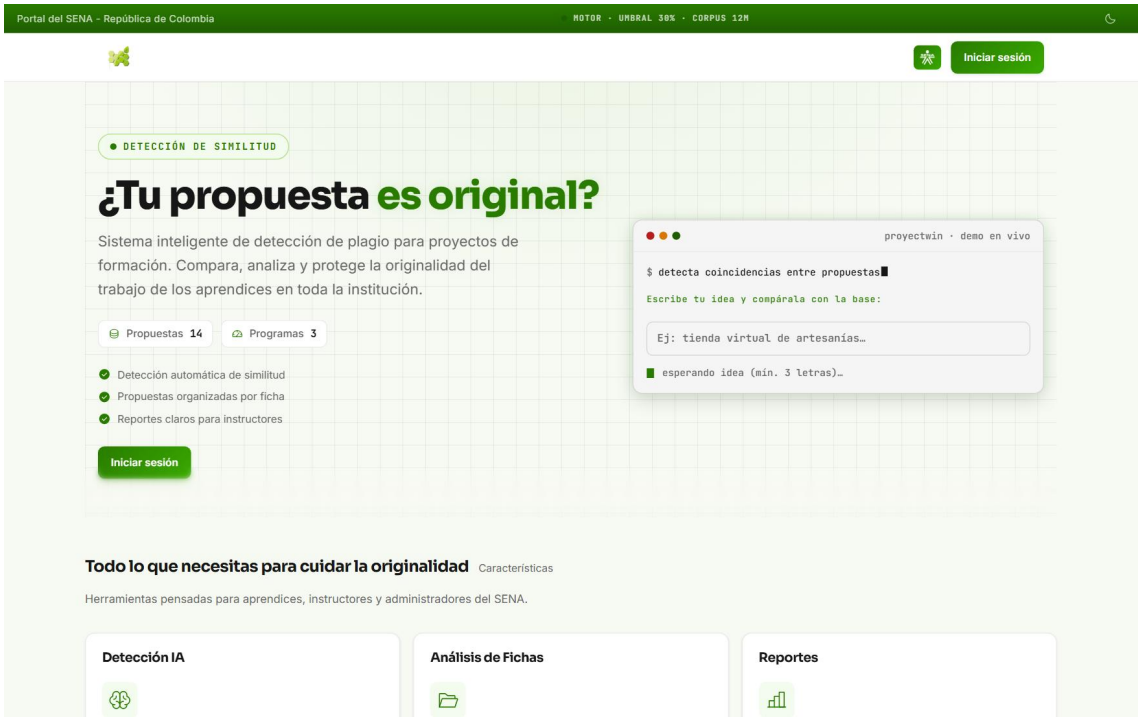
Excluido o no evidenciado: aplicación Android como código del frontend, backend Laravel, correo real de producción, monitoreo APM, certificaciones ISO, medición de Core Web Vitals, cobertura de pruebas y cualquier módulo no presente en src/. Las capturas disponibles son evidencia histórica de la interfaz; no se tomaron capturas nuevas porque no se ejecutó el sistema completo en esta auditoría.

5. Descripción del frontend

La SPA se carga una vez y navega en cliente con React Router. Las páginas se cargan de forma diferida (lazy) con Suspense y un fallback accesible. ProtectedRoute valida la sesión y el rol; SafeRoute envuelve la página con un ErrorBoundary y acciones de recuperación. La sesión se rehidrata con /auth/me al arrancar.

La interfaz usa CSS Modules, tokens de diseño y componentes reutilizables. El tema claro/oscuro se conserva en el contexto de tema. El portal público consume endpoints abiertos de catálogos, resumen y demo. Las páginas privadas consumen recursos autenticados y respetan el alcance que impone el backend.

Figura 1. Portal público de ProyecTwin (captura de referencia)



Fuente: Captura existente en docs/manual-tecnico (16-oct-2026); la estructura de navegación se verificó contra App.jsx. Los datos mostrados son de demostración.

5.1 Funcionalidades implementadas

Tabla 4. Funcionalidades del frontend

Módulo	Funcionalidad	Evidencia
Portal público	Landing, demo, catálogos, resumen y acceso.	pages/public/Home; lib/recursos.js
Autenticación	Login por correo, recuperación, restablecimiento y activación.	pages/public/Login, RecuperarContrasena, RestablecerContrasena, ActivarCuenta
Sesión	Cookie httpOnly, /auth/me, cierre y expiración.	contexts/AuthContext; lib/api.js
Aprendiz	Dashboard, propuestas, ficha, similitudes, alertas, perfil y soporte.	pages/aprendiz/*
Instructor	Dashboard, revisión, fichas, similitudes, alertas y perfil.	pages/instructor/*
Admin	Usuarios, catálogos, fichas, propuestas, similitudes, bitácora, soporte, notificaciones y motor.	pages/admin/*
Navegación	Rutas por rol, redirecciones y 404.	App.jsx; components/ProtectedRoute, SafeRoute
Datos	Servicios por recurso, caché de GET y CSRF.	lib/api.js; lib/recursos.js
Formularios	Campos, validación, errores 422 y estados de envío.	components/FormField, Input; utils/validation
Accesibilidad	Foco, ARIA, teclado y pruebas axe en E2E.	useFocusTrap; E2E; tests
Manual y APK	Manual de usuario y botón APK condicionado a URL HTTPS; hoy no se publica.	pages/public/Home; ApkDownloadButton

Fuente: frontend/src; package.json.

6. Requisitos de hardware y software

Tabla 5. Requisitos para ejecutar el frontend

Recurso	Requisito verificado	Observación
Sistema operativo	Windows, Linux o macOS con Node y navegador moderno.	El entorno de auditoría es Windows.
Node.js	^20.19.0 || ^22.12.0 || >=24.0.0; local 24.19.0.	Vite 8 exige este rango.
npm	10+; local 11.17.0.	Se usa npm ci en CI.
RAM / disco	No declarado; build Vite y node_modules requieren espacio.	No hay requisito formal.
Navegador	Chrome/Edge/Firefox modernos; Playwright usa Chromium.	Sin matriz formal de navegadores.
Backend	API en 127.0.0.1:8000 para desarrollo; proxy Vite.	Sin backend, las páginas privadas fallan.
Correo	Mailpit para probar activación/recuperación.	Opcional.
Playwright	Chromium instalado para E2E.	npx playwright install chromium.

Fuente: package.json, package-lock.json, vite.config.js y README.

7. Tecnologías y versiones exactas

Las versiones se obtuvieron de package-lock.json y del entorno. No se confunde la restricción declarada con la versión resuelta. React se declara ^19.2.5 y se resuelve 19.2.6; Vite se declara ^8.0.10 y se resuelve 8.2.2.

Tabla 6. Tecnologías y versiones del frontend

Tecnología	Versión	Fuente	Estado
React	19.2.6	package-lock.json (declarado ^19.2.5)	Resuelta
React DOM	19.2.6	package-lock.json	Resuelta
React Router DOM	7.18.3	package-lock.json (declarado ^7.15.1)	Resuelta
Vite	8.2.2	package-lock.json (declarado ^8.0.10)	Resuelta
Vitest	4.1.11	package-lock.json (declarado ^4.1.11)	Resuelta
Playwright	1.62.1	package-lock.json	Resuelta
ESLint	10.3.0	package-lock.json (declarado ^10.2.1)	Resuelta
phosphor-react	1.4.x	package.json	Declarada
@testing-library/react	16.3.3	package-lock.json	Resuelta
Node.js	24.19.0 local; requiere ^20.19.0 || ^22.12.0 || >=24.0.0	node --version; package-lock.json	Entorno/requisito
npm	11.17.0 local	npm --version	Entorno

Fuente: package.json, package-lock.json y entorno local.

PENDIENTE DE VERIFICACIÓN: la versión exacta de Node/npm en producción (Docker) es 22.23.4/la que incluya la imagen; el Dockerfile fija Node 22.23.4. La versión de npm dentro de la imagen no se declara.

7.1 Dependencias principales

Tabla 7. Dependencias del frontend

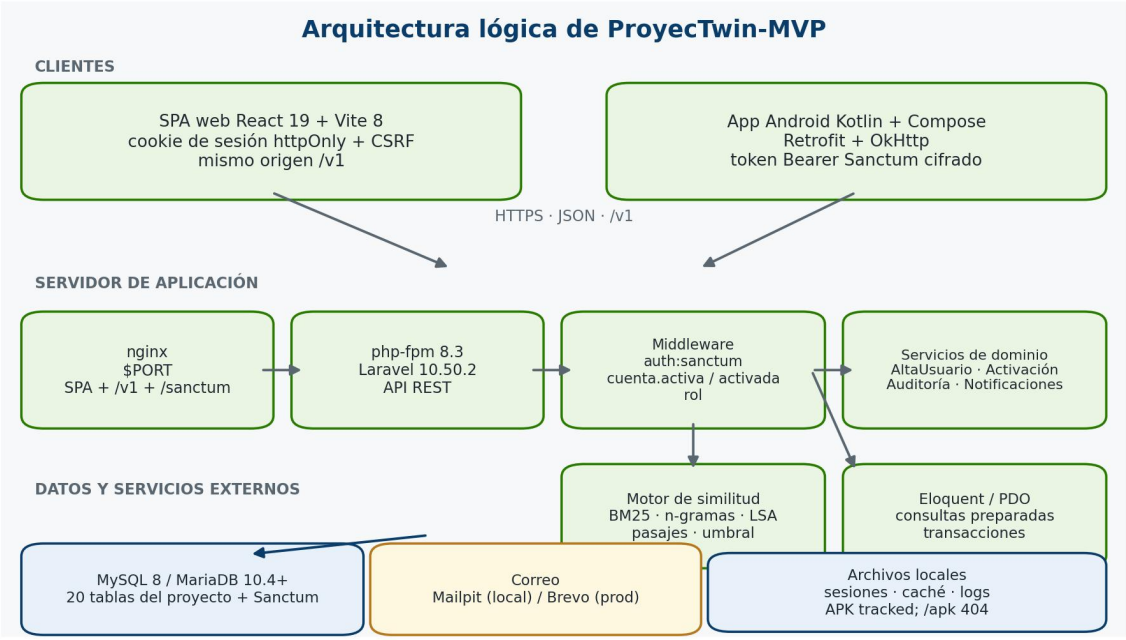
Paquete	Versión resuelta	Uso
react / react-dom	19.2.6	UI y renderizado.
react-router-dom	7.18.3	Rutas y navegación.
phosphor-react	1.4.x	Iconografía.
vite	8.2.2	Build y servidor.
vitest	4.1.11	Pruebas unitarias.
@testing-library/react	16.3.3	Pruebas de componentes.
@playwright/test	1.62.1	E2E y accesibilidad.
eslint	10.3.0	Lint.
jsdom	29.1.1	Entorno DOM para Vitest.

Fuente: package-lock.json.

8. Arquitectura

La SPA es una aplicación de una sola página con rutas declarativas, carga diferida y separación por rol. La arquitectura se organiza en capas: entrada (main.jsx/App.jsx), páginas, layouts, componentes, contextos, servicios de API, utilidades y tokens de diseño. No hay Redux ni Zustand: el estado global se limita a sesión, tema y configuración del motor; el resto es estado local de página/componente.

Figura 2. Arquitectura lógica del sistema y posición de la SPA



Fuente: Elaboración propia a partir de Dockerfile, App.jsx y lib/api.js.

Tabla 8. Capas de la SPA

Capa	Responsabilidad	Archivos
Entrada	Montar React, providers y rutas.	src/main.jsx; src/App.jsx
Páginas	Vistas por rol y flujos.	src/pages/public, aprendiz, instructor, admin
Layouts	Estructura visual y navegación.	src/layouts/*
Componentes	Piezas reutilizables y base.	src/components/*
Contextos	Sesión, tema y motor.	src/contexts/*
Servicios	Consumo de API y caché.	src/lib/api.js; src/lib/recursos.js
Utilidades	Validación, fechas, fotos, tokens y estilos.	src/utils; src/tokens

Fuente: frontend/src.

Flujo de una petición: usuario → componente/página → servicio de recursos → apiFetch → proxy Vite/nginx → backend → respuesta JSON → estado local/contexto → render. Si la respuesta es 401, se emite el evento de sesión expirada y ProtectedRoute redirige al login. Si es 419, se renueva el CSRF y se reintenta una vez.

8.1 Decisión de mismo origen

En desarrollo, Vite proxya /v1 y /sanctum a 127.0.0.1:8000; en producción, nginx sirve la SPA y la API en el mismo origen. Esto evita el preflight CORS en cada petición autenticada y permite la cookie httpOnly de Sanctum. La configuración usa 127.0.0.1 y no localhost para evitar resoluciones IPv6 en Windows.

```
const API = process.env.PROYECTWIN_API || 'http://127.0.0.1:8000'
const proxy = {
  '/v1': { target: API, changeOrigin: true },
  '/sanctum': { target: API, changeOrigin: true },
}
export default defineConfig({ plugins: [react()], server: { proxy }, preview: { proxy } })
```

Proxy 1 — Archivo: frontend/vite.config.js

9. Estructura de carpetas

```
frontend/
├── src/
│   ├── components/     # componentes reutilizables y *Base
│   ├── pages/          # public, aprendiz, instructor, admin
│   ├── layouts/        # AuthLayout, DashboardLayout, LandingLayout
│   ├── contexts/       # AuthContext, ThemeContext, MotorContext
│   ├── hooks/          # useFocusTrap
│   ├── lib/            # api.js, recursos.js, errores.js, urlApk.js
│   ├── constants/      # routes, estados, paginación
│   ├── utils/          # validación, fechas, fotos, enlaces
│   ├── tokens/         # tokens de diseño CSS
│   ├── test/           # setup de Vitest
│   ├── App.jsx         # rutas y control de acceso
│   └── main.jsx        # montaje y providers
├── e2e/                # 11 specs Playwright y utilidades
├── public/             # imágenes, manual, APK y assets
├── package.json / package-lock.json
├── vite.config.js / vitest.config.js / playwright.config.js
└── eslint.config.js
```

Estructura 1 — Archivo: frontend/ (árbol resumido)

Observación: el backend tiene su propio package.json y Vite 4 para assets de Laravel; no debe confundirse con la SPA, que vive en frontend/ y usa Vite 8.

10. Componentes

La carpeta components contiene piezas reutilizables y variantes *Base compartidas entre roles. La tabla resume los componentes de mayor impacto; no es un inventario exhaustivo de cada archivo CSS.

Tabla 9. Componentes principales

Componente	Función
ProtectedRoute	Valida sesión y rol antes de renderizar una página.
SafeRoute	ErrorBoundary con acciones de recuperación y soporte.
DashboardLayout / LandingLayout / AuthLayout	Estructuras visuales por contexto.
DataTable / Pagination / FilterBar	Tablas, paginación y filtros.

Componente	Función
FormField / Input / FormularioBase	Campos accesibles y validación.
DashboardMetrics / MetricCard / Charts	Indicadores y gráficos.
SimilarityCards / ScoreDial / SimilitudesHistoricas	Presentación de similitudes.
DetalleProyectoBase / DetalleFichaBase / DetalleSimilitudBase	Vistas compartidas entre roles.
ObservacionHilo / HistorialProyecto	Hilos de observaciones e historial.
ApkDownloadButton / AppUpdateNotice	Descarga y aviso de actualización Android.
ErrorBoundary / PageFallback / EmptyState / Skeleton	Estados de error, carga y vacío.
Lightbox / Tooltip / Tag / StatusMark / GradeBadge	Detalles de interfaz.

Fuente: frontend/src/components.

11. Páginas y módulos

Hay 48 componentes de página (archivos .jsx distintos de index) y 55 declaraciones de ruta en App.jsx: 48 páginas y 7 redirecciones heredadas. La tabla agrupa las rutas por experiencia; la sección 31.1 reproduce el inventario completo.

Tabla 10. Rutas representativas por rol

Rol	Rutas	Propósito
Público	/, /login, /recuperar-contrasena, /restablecer-contrasena, /activar-cuenta	Portal, acceso y recuperación.
Aprendiz	/aprendiz/dashboard, propuestas, ficha, similitudes, alertas, perfil, detalle-*	Gestión de propuesta, ficha y similitud.
Instructor	/instructor/dashboard, revision-propuestas, fichas, similitudes, alertas, perfil	Revisión y gestión de fichas.
Admin	/admin/dashboard, usuarios, redes-conocimiento, config-similitud, fichas, bitacora, proyectos, similitudes, reportes-fallas, notificaciones, perfil	Administración y supervisión.
Compatibilidad	/aprendiz/mis-proyectos, /instructor/gestionar-fichas, /admin/gestion-usuarios, *	Redirecciones heredadas y 404.

Fuente: frontend/src/App.jsx.

Módulo destacado — propuestas: el aprendiz crea un borrador, selecciona equipo, envía y consulta el resultado del análisis. El instructor revisa, aprueba o rechaza con observación; el admin supervisa. Las páginas usan DetalleProyectoBase y recursos.proyectos para mantener un contrato único.

Módulo destacado — similitudes: lista, detalle y comparación de pares; se muestran porcentaje, señales y pasajes. El cálculo es del backend; la SPA solo presenta y permite recalcular al admin.

12. Sistema de navegación

App.jsx declara las rutas y aplica ProtectedRoute con allowedRoles. Las rutas de compatibilidad redirigen a las rutas nuevas con Navigate. Las rutas desconocidas muestran PaginaNoEncontrada. ScrollToTop y viewTransition mejoran la experiencia de navegación.

Tabla 11. Control de acceso por rol

Ruta / grupo	Roles permitidos	Comportamiento
Públicas	Sin sesión	Renderizan portal, login, recuperación y activación.
/aprendiz/*	aprendiz	Redirige a login si no hay sesión; AccesoDenegado si el rol no coincide.
/instructor/*	instructor	Ídem; algunas vistas de compañero permiten aprendiz/instructor.
/admin/*	admin	Ídem; el backend vuelve a autorizar cada operación.
*	Público	Página 404.

Fuente: App.jsx; ProtectedRoute; constants/routes.js.

```
export default function ProtectedRoute({ allowedRoles, children }) {
  const { user, isAuthenticated, cargando } = useAuth()
  if (cargando) return <p role="status" aria-busy="true">Cargando sesión…</p>
  if (!isAuthenticated) return <Navigate to="/login" replace />
  if (allowedRoles && !allowedRoles.includes(user.rol)) return <AccesoDenegado
role={user.rol} />
  return children
}
```

Navegación 1 — Archivo: frontend/src/components/ProtectedRoute/ProtectedRoute.jsx

13. Gestión del estado

El estado global es mínimo: AuthContext (usuario y sesión), ThemeContext (tema claro/oscuro) y MotorContext (configuración pública del motor). El resto del estado vive en cada página/componente. apiFetch mantiene una caché TTL de GET y una mapa de peticiones en vuelo para deduplicar.

Tabla 12. Estado y persistencia

Estado	Dónde vive	Persistencia
Sesión	AuthContext + cookie httpOnly del navegador.	Cookie; se rehidrata con /auth/me.
Tema	ThemeContext + localStorage.	Sí, preferencia local.
Motor	MotorContext.	Se consulta a la API.
Datos de página	useState/useEffect o hooks locales.	No persiste al recargar.
Caché de GET	Map en memoria de api.js.	TTL corto; se invalida tras escrituras.
Formularios	Estado local del componente.	No persiste.

Fuente: frontend/src/contexts; frontend/src/lib/api.js.

Riesgo: la caché en memoria no se comparte entre pestañas y se pierde al recargar; es deliberada para evitar datos obsoletos. Las escrituras invalidan la caché antes y después de la petición.

14. Formularios y validaciones

Los formularios usan componentes controlados, labels asociados, mensajes de error y validación en cliente. La validación definitiva está en el backend; los errores 422 se mapean a campos con toFieldErrors. La política de contraseña del cliente se alinea con la del backend (mínimo 12).

Tabla 13. Validaciones del cliente

Formulario	Validación	Evidencia
Login	Correo requerido y formato email; contraseña requerida.	pages/public/Login; utils/validation
Activación	Token, contraseña y confirmación; política de 12.	pages/public/ActivarCuenta
Recuperación	Correo válido y token.	RecuperarContrasena; RestablecerContrasena
Propuesta	Título, resumen, objetivos y campos obligatorios.	pages/aprendiz/Propuestas
Perfil	Nombre, correo y foto; límites de imagen.	PerfilBase; utils/foto
Soporte	Tipo, motivo y descripción.	ReportarFallaBase

Fuente: frontend/src/pages y utils.

Observación: el cliente puede mostrar validaciones inmediatas, pero no es autoridad. Un payload manipulado debe ser rechazado por el backend; las pruebas negativas de API cubren esa expectativa.

15. Servicios

La capa de servicios se divide en api.js (transporte, CSRF, caché, errores) y recursos.js (funciones por recurso). Las páginas no llaman a fetch directamente: usan los servicios, lo que facilita mantener el contrato y sustituir la implementación.

Tabla 14. Servicios por recurso

Servicio	Operaciones principales
usuarios	Listar, paginar, obtener, perfil, crear, actualizar, reenviar activación, restablecer, eliminar.
redes / programas	Catálogos institucionales.
instructores / aprendices	Perfiles, ficha y roster.
fichas	Listar, obtener, crear, actualizar, agregar aprendiz, eliminar.
proyectos	Listar, obtener, crear, enviar, historial, actualizar, eliminar y equipo.
similitudes	Listar, obtener, recalcular y eliminar.
motor / demo	Configuración pública, resumen y comparación demo.
appVersion	Metadatos de la versión Android.
notificaciones / reportes / observaciones / auditoría / métricas	Recursos de soporte y administración.

Fuente: frontend/src/lib/recursos.js.

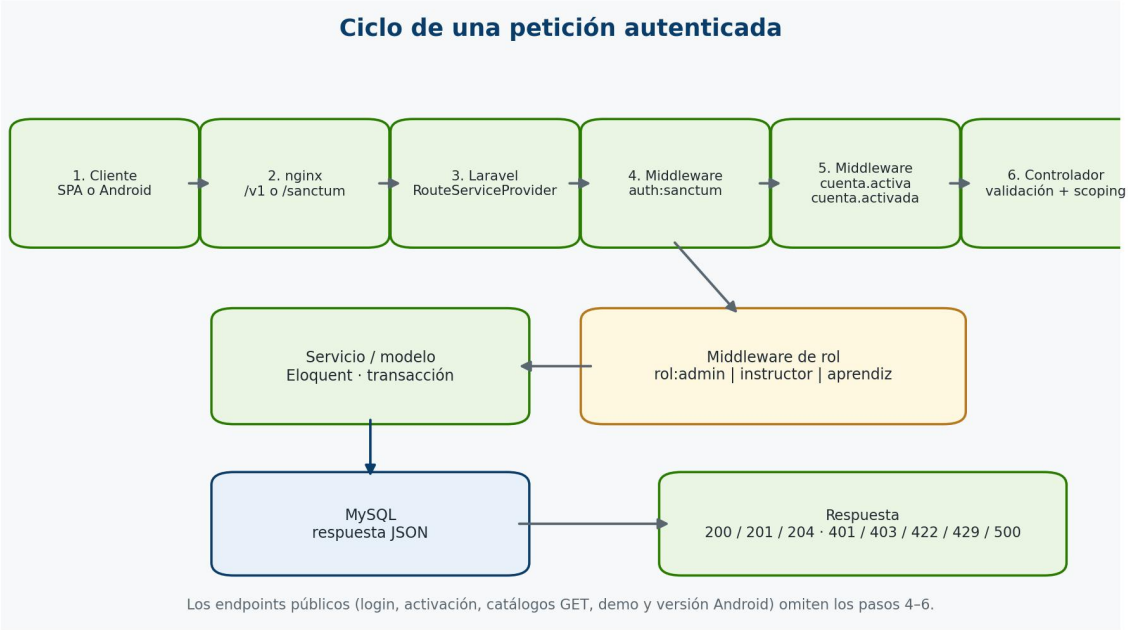
16. Integración con backend

Todas las peticiones usan JSON y Accept: application/json. Las escrituras incluyen el token CSRF leído de la cookie XSRF-TOKEN. Si el backend responde 419, apiFetch renueva la cookie y reintenta una sola vez. Si responde 401, emite un evento global para limpiar la sesión. Si responde 403 con code cuenta_no_activada, también limpia la sesión.

```
export async function apiLogin(correo, password, remember = false) {
  const valor = String(correo || '').trim().toLowerCase()
  return apiFetch('/auth/login', {
    method: 'POST',
    body: { correo: valor, password, recordarme: !!remember },
    auth: false,
  })
}
```

Integración 1 — Archivo: frontend/src/lib/api.js

Figura 3. Ciclo de una petición autenticada



Fuente: Elaboración propia a partir de app/Http/Kernel.php, lib/api.js y vite.config.js.

Tabla 15. Flujo de datos

Paso	Componente	Resultado
1. Usuario interactúa	Página/componente	Evento y estado local.
2. Servicio	recursos.js	Construye ruta y parámetros.
3. Transporte	api.js	CSRF, fetch, timeout y caché.
4. Proxy/nginx	Vite o nginx	Mismo origen hacia Laravel /v1.
5. Backend	Laravel	Valida, autoriza, consulta y responde JSON.
6. Render	Página/contexto	Actualiza estado, mensajes y navegación.

Fuente: frontend/src/lib; vite.config.js; docker/nginx.conf.

17. Variables de entorno

Tabla 16. Variables del frontend

Variable	Valor por defecto	Uso	¿Expuesta al navegador?
VITE_API_URL	/v1	Prefijo de la API; se compila en el bundle.	Sí, por diseño de Vite.
PROYECTWIN_API	http://127.0.0.1:8000	Target del proxy Vite; no se inyecta en el bundle.	No.
E2E_DB_DATABASE / E2E_ALLOW_RESET	Solo E2E	Guardas del entorno de pruebas.	No en producción.
PROYECTWIN_API (E2E)	http://127.0.0.1:8001	Backend E2E.	No en producción.

Fuente: frontend/.env.example; vite.config.js; playwright.config.js.

Diferencia obligatoria: `VITE_API_URL` es una variable del cliente Vite y se incorpora al bundle; su valor por defecto es `/v1`. `PROYECTWIN_API` no es una variable `VITE_*`: se lee en `vite.config.js` como variable del proceso para dirigir el proxy de desarrollo/preview y no se incluye en el bundle. No son equivalentes ni intercambiables.

Advertencia: cualquier variable VITE_ *se incorpora al bundle y es pública. No se deben usar para secretos, claves de API ni credenciales. El proyecto no contiene secretos en VITE_*.

18. Autenticación

La SPA no guarda el token en localStorage. La autenticación se apoya en una cookie httpOnly emitida por Sanctum; el navegador la envía automáticamente. Al arrancar, AuthContext llama a /auth/me para reconstruir la sesión. Al cerrar sesión, llama a /auth/logout y limpia el estado.

Tabla 17. Flujo de autenticación web

Paso	Llamada	Resultado
1. Login	POST /v1/auth/login	Cookie httpOnly + usuario/rol.
2. Rehidratación	GET /v1/auth/me	Usuario mínimo en memoria.
3. Navegación	ProtectedRoute	Redirección por rol.
4. Expiración	401 en cualquier llamada	Evento de sesión expirada y limpieza.
5. Logout	POST /v1/auth/logout	Cookie invalidada y estado limpio.

Fuente: AuthContext; lib/api.js; AuthController.

No hay registro público: las cuentas las crea el administrador y se activan por correo. Esta decisión se refleja en la UI y en el contrato del backend.

19. Manejo de errores

apiFetch normaliza los errores: status 0 para timeout/sin conexión, 4xx/5xx con message y errors. Las páginas muestran mensajes claros y conservan el formulario para corregir. ErrorBoundary evita que un error de render deje la aplicación en blanco; SafeRoute ofrece volver al dashboard o reportar soporte.

Tabla 18. Errores del cliente

Situación	Tratamiento	Evidencia
401	Evento de sesión expirada; redirección a login.	AuthContext; ProtectedRoute
403 cuenta_no_activada	Limpieza de sesión y mensaje.	lib/api.js
403 de permisos	Mensaje de acceso denegado sin cerrar sesión.	ProtectedRoute; AccesoDenegado
419 CSRF	Renovación y reintento único.	lib/api.js
422 validación	Mapeo a campos del formulario.	toFieldErrors
Timeout	Mensaje de servidor lento; status 0.	lib/api.js
Error de render	ErrorBoundary con acciones.	SafeRoute; ErrorBoundary

Fuente: frontend/src/lib; componentes de error.

20. Instalación

La SPA requiere Node 20.19+/22.12+/24+, npm y el backend disponible. Los comandos siguientes son los del repositorio; no se inventan scripts. `npm test` ejecuta Vitest; `npm run test:e2e` ejecuta el wrapper Playwright con base desechable.

```
cd frontend
npm ci
npm run dev          # Vite en http://localhost:5173
# En otra terminal:
```

```
cd ../backend
php artisan serve --host=127.0.0.1 --port=8000
```

Instalación 1 — Archivo: frontend/README.md

No es necesario crear frontend/.env: /v1 es el valor predeterminado. Solo se define VITE_API_URL si se quiere cambiar el prefijo. Para apuntar el proxy a otro backend sin modificar el bundle se define PROYECTWIN_API en el proceso de Vite; por ejemplo, en PowerShell: `$env:PROYECTWIN_API='http://127.0.0.1:8000'`. El backend debe estar operativo y con una cuenta activa; su instalación, migraciones y base de datos pertenecen al manual técnico del backend y no se repiten aquí. Para E2E se requiere Mailpit, MySQL y el wrapper del backend.

21. Configuración

La configuración relevante está en vite.config.js (proxy), vitest.config.js (jsdom y setup), playwright.config.js (preview 4173, un worker, guardas E2E) y eslint.config.js (flat config con globals de navegador y Node). No hay archivo de configuración de producción específico: el build usa VITE_API_URL.

Tabla 19. Archivos de configuración

Archivo	Función
vite.config.js	Proxy /v1 y /sanctum; build de la SPA.
vitest.config.js	jsdom, setup y patrón de tests.
playwright.config.js	E2E, preview 4173, workers 1 y guardas.
eslint.config.js	Flat config, globals browser/Node.
package.json	Scripts dev, build, lint, test, test:e2e, quality.
playwright.config.js	Un solo archivo de configuración E2E; no existe playwright.volumen.config.js.

Fuente: frontend/.

Corrección respecto a v2.0: el manual anterior mencionaba `playwright.volumen.config.js`, `test:unit`, `test:agentes*` y que `npm test` ejecutaba Playwright. El repositorio actual no contiene esos archivos/scripts: `npm test` ejecuta Vitest y `npm run test:e2e` ejecuta el wrapper E2E.

22. Ejecución local

Para desarrollo se necesitan dos terminales: backend en 8000 y frontend en 5173. Vite proxyfica /v1 y /sanctum. El login por correo requiere una cuenta activa; la activación y recuperación se prueban con Mailpit. Las rutas privadas redirigen a /login si no hay sesión.

- Verificar que el backend responda /v1/config-similitud.
- Abrir http://localhost:5173 y comprobar el portal.
- Iniciar sesión con una cuenta activa; comprobar redirección por rol.

- Probar una ruta protegida sin sesión; debe redirigir a login.
- Probar un 422 con un formulario inválido; los mensajes deben mapearse al campo.
- Revisar la consola del navegador: no debe haber errores de CORS ni 419 repetidos.

23. Compilación

El build de producción se genera con Vite en dist/ y se sirve con nginx o `npm run preview`. La API se consume con ruta relativa /v1 en producción. El build no incluye secretos; VITE_API_URL se incorpora como parte del bundle.

```
npm run lint
npm test
npm run build
npm run preview
```

Build 1 — Archivo: frontend/package.json

PENDIENTE DE VERIFICACIÓN: no se ejecutó el build en este entorno porque no había node_modules/. La CI histórica registra build correcto en el commit 99806b5; no se afirma el estado del commit actual.

24. Pruebas

La estrategia combina Vitest + React Testing Library para unitarias, Playwright para E2E y ESLint/build como gates estáticos. Hay 32 archivos de prueba y 90 definiciones estáticas de `it/test`; los documentos del proyecto citan 90/90 y 110 tests en momentos distintos, y el manual v2.0 citaba 98/98. Esas cifras no están reconciliadas y no se presentan como el resultado actual.

Tabla 20. Evidencia de pruebas del frontend

Tipo	Archivos / alcance	Estado en esta auditoría
Unitarias Vitest	32 archivos; componentes, páginas, hooks, utils y API.	No ejecutadas; sin node_modules.
E2E Playwright	11 specs; 26 escenarios estáticos (25/25 histórico) (acceso, usuarios, fichas, propuestas, similitud, soporte, correo).	No ejecutadas; requieren backend/base/Mailpit.
Lint	ESLint 10 con flat config; cubre e2e y playwright*.config.js.	No ejecutado; CI histórica.
Build	Vite 8.	No ejecutado; CI histórica.
Accesibilidad	Axe en E2E; pruebas de teclado/foco en unitarias.	Parcial; no reejecutada.

Fuente: frontend/src/**/*.test.*; frontend/e2e; package.json; CI.md.

Evidencia histórica: CI.md registra 9 corridas exitosas al 2-nov-2026; la última referenciada es la run 41572290608 sobre el commit 99806b5. El gate E2E 6AP registró 25/25 y el inventario 6AL citó 90/90 unitarias. Son evidencias de momentos anteriores, no del commit 1ebab35. No se declaran pruebas actuales verdes.

25. Seguridad

La seguridad del cliente se apoya en la cookie httpOnly, CSRF, escape de React, rutas protegidas y validación. El cliente no es autoridad: el backend vuelve a autorizar cada operación. No se usan dangerouslySetInnerHTML en el código revisado. Las variables VITE_* son públicas por diseño.

Tabla 21. Controles de seguridad del frontend

Control	Implementación	Estado
Sesión	Cookie httpOnly; no se guarda token en localStorage.	Implementado
CSRF	XSRF-TOKEN + X-XSRF-TOKEN; reintento 419.	Implementado
XSS	Escape de React; sin dangerouslySetInnerHTML revisado.	Implementado
Autorización UI	ProtectedRoute + backend autoritativo.	Implementado
Secretos	Sin claves en VITE_*; .env no versionado.	Implementado
Cabeceras	CSP/HSTS/X-Frame-Options desde nginx.	Implementado en servidor
Dependencias	Sin SCA/auditoría automática.	Parcial
Accesibilidad	Axe E2E y pruebas de foco; no reejecutado.	Parcial
Pruebas de seguridad	Sin pentest ni pruebas de abuso dedicadas.	No implementado

Fuente: frontend/src; docker/nginx.conf.

Riesgos residuales: XSS depende de datos y dependencias; no hay CSP nonce; no hay monitoreo de errores en producción; la sesión cross-site requeriría SameSite=None y HTTPS, pero el despliegue previsto es same-origin. No se declara cumplimiento de ISO/IEC 27001.

26. Calidad y normas ISO — Frontend

Alcance normativo. Las normas se usan como referencia y marco de análisis. Este documento identifica buenas prácticas, controles implementados y brechas; no demuestra conformidad formal ni certificación. La conformidad requeriría una auditoría independiente, un sistema de gestión documentado y evidencias operativas sostenidas. ProyecTwin-MVP no está certificado bajo ISO/IEC 27001, ISO 9001 ni ninguna otra norma.

26.1 Normas y ediciones consideradas

Se adoptan como marco de análisis las ediciones contrastadas en ISO.org el 4 de noviembre de 2026: ISO/IEC 25010:2023, ISO/IEC/IEEE 12207:2026, ISO/IEC/IEEE 29148:2018 (en revisión), ISO/IEC 27001:2022 + Amd 1:2024, ISO 9001:2026, ISO/IEC 25023:2016 (en revisión) y la familia ISO/IEC/IEEE 29119 (partes 1:2022, 2:2021, 3:2021, 4:2021 y 5:2024). No se citan numerales ni textos normativos protegidos; se usan las características y procesos públicos como marco de referencia. La vigencia de una edición no implica conformidad del proyecto.

ISO/IEC 25010:2023 sustituyó la edición 2011 y amplió el modelo a nueve características: adecuación funcional, eficiencia de desempeño, compatibilidad, capacidad de interacción, fiabilidad, seguridad, mantenibilidad, flexibilidad y seguridad funcional (*safety*). Las

denominaciones *capacidad de interacción* y *flexibilidad* sustituyen a *usabilidad* y *portabilidad*. En este manual se evalúan las ocho primeras y se justifica la novena cuando no aplica.

26.2 ISO/IEC 25010 — evaluación por característica

Tabla 22. Evaluación ISO/IEC 25010:2023 del frontend web

Característica	Evidencia técnica	Estado
Adecuación funcional	55 rutas y 48 componentes de página cubren portal, autenticación, tres roles, propuestas, fichas, similitudes, alertas, soporte, bitácora y administración. 32 archivos de prueba y 11 specs E2E documentados.	Parcial
Eficiencia de desempeño	Lazy loading de páginas, caché TTL de GET, deduplicación de peticiones, paginación y build Vite. Sin mediciones Lighthouse/Core Web Vitals en el commit actual.	No verificable
Compatibilidad	Navegadores modernos con React 19; proxy same-origin; diseño responsive y modo oscuro; integración Android separada. Sin matriz formal de navegadores.	Parcial
Capacidad de interacción	Formularios con labels, foco, teclado, roles ARIA, ErrorBoundary y pruebas axe en E2E. Accesibilidad no reejecutada en este commit.	Parcial
Fiabilidad	Manejo de 401/403/419/422, reintento CSRF, caché invalidada tras escrituras, ErrorBoundary y SafeRoute. Sin pruebas de resiliencia de red.	Parcial
Seguridad	Cookie httpOnly, CSRF, escape de React, sin secretos VITE_*, rutas protegidas por rol y CSP del servidor. El cliente no es autoridad; faltan pruebas de seguridad específicas.	Parcial
Mantenibilidad	Componentes reutilizables, CSS Modules, contextos, hooks, ESLint 10 y Vitest; CI frontend. Sin cobertura ni Storybook.	Parcial
Flexibilidad	Variables VITE_*, proxy configurable, rutas centralizadas, temas y componentes base. Sin pruebas de despliegue alternativo.	Parcial
Safety (seguridad funcional)	No aplica: la interfaz no controla procesos físicos ni decisiones críticas.	No aplicable

Fuente: Elaboración propia a partir del código, la configuración y las pruebas existentes.

26.3 ISO/IEC/IEEE 29148 — requisitos y trazabilidad

Los requisitos se clasifican con los estados **implementado**, **parcialmente implementado**, **no implementado** y **pendiente de validación**. La clasificación se apoya en rutas, controladores, rutas, controladores, pruebas y documentación; no se inventan requisitos ni resultados.

Tabla 23. Catálogo de requisitos verificables del frontend web

ID	Requisito	Evidencia	Estado
RF-01	Portal público con información, demo y acceso.	Home.jsx; api.demo; Home.test	Implementado
RF-02	Login por correo, recuperación y activación de cuenta.	Login, RecuperarContrasena, RestablecerContrasena, ActivarCuenta	Implementado
RF-03	Navegación protegida por rol con redirección y acceso denegado.	App.jsx; ProtectedRoute; SafeRoute; routes.test	Implementado
RF-04	Dashboard y módulos de aprendiz, instructor y admin.	48 componentes en src/pages	Implementado
RF-05	Formularios con validación y mensajes de error 422.	FormField, Input, utils/validation; api.toFieldErrors	Implementado
RF-06	Consumo de API same-origin con CSRF y reintento 419.	lib/api.js; vite.config.js	Implementado
RF-07	Caché de GET, deduplicación e invalidación tras escrituras.	lib/api.js; api.test.js	Implementado
RF-08	Modo oscuro y diseño responsive.	ThemeContext; CSS Modules; capturas	Implementado
RF-09	Accesibilidad básica y foco atrapado en diálogos.	useFocusTrap; roles ARIA; pruebas axe E2E	Parcial
RF-10	Manejo de sesión expirada y cuenta suspendida/no activada.	AuthContext; ProtectedRoute; api.js	Implementado
RF-11	Publicar y enlazar el manual de usuario y la descarga Android.	Home/ApkDownloadButton; manual sí; APK no publicado y botón oculto sin URL HTTPS.	Parcial
RF-12	Pruebas unitarias y build en CI.	vitest.config.js; ci.yml	Implementado
RF-13	Cobertura de integración frontend-backend en Vitest.	Solo unitarias; E2E separado	Parcial
RF-14	Medir Core Web Vitals y accesibilidad en cada release.	Sin Lighthouse ni reporte automatizado	No implementado

ID	Requisito	Evidencia	Estado
RF-15	Validar la suite actual en el commit de referencia.	Sin node_modules en el entorno	Pendiente de validación

Fuente: Elaboración propia; el estado refleja la evidencia disponible al commit de referencia.

26.4 ISO/IEC/IEEE 12207 — procesos del ciclo de vida

El mapeo se limita a procesos y actividades con evidencia en el repositorio. No se afirma una implantación formal de ISO/IEC/IEEE 12207:2026 ni se evalúa la madurez de la organización.

Tabla 24. Procesos y actividades evidenciados

Proceso / actividad	Evidencia	Estado	Observación
Planeación	README, docs y commits.	Parcial	Sin plan formal.
Requisitos	Catálogo de este manual; documentación dispersa.	Parcial	Aprobar baseline.
Diseño	Componentes, layouts y App.jsx.	Implementado	Mantener diagramas.
Implementación	Código React y CSS Modules.	Implementado	Sin revisión obligatoria.
Integración	API same-origin y proxy.	Implementado	E2E no actual.
Verificación	Vitest, Playwright, ESLint, build.	Parcial	Sin ejecución actual.
Transición	Build Vite y nginx.	Parcial	No verificado en producción.
Operación	Portal, manual y APK.	Parcial	Sin APM.
Mantenimiento	Dependencias y documentación.	Parcial	Falta calendario.
Disposición	No aplica al MVP.	No aplicado	Sin política de datos.

Fuente: Elaboración propia.

26.5 ISO/IEC 27001 — controles y buenas prácticas de seguridad

Se contrastan prácticas técnicas observables con los temas de control de ISO/IEC 27001:2022. La tabla no atribuye números de control ni declara un SGSI implantado; distingue control implementado, parcial y ausente.

Tabla 25. Prácticas de seguridad observadas

Tema / práctica	Evidencia	Estado	Brecha o recomendación
Sesión y CSRF	Cookie httpOnly, XSRF-TOKEN, reintento 419.	Implementado	Mantener pruebas de expiración.
Autorización UI	ProtectedRoute y backend autoritativo.	Implementado	Probar rutas y roles negativos.
XSS	Escape de React; sin HTML crudo revisado.	Parcial	Añadir CSP estricta y revisión de dependencias.
Secretos	Sin VITE_* sensibles.	Implementado	Revisar en cada build.
Dependencias	Sin auditoría automática.	No implementado	Incorporar npm audit/SCA.
Errores	Mensajes claros; no se exponen trazas del backend.	Implementado	Monitorear errores en producción.
Accesibilidad	ARIA, foco, axe E2E.	Parcial	Reejecutar y ampliar cobertura.
Privacidad	No se registran tokens; datos visibles según backend.	Parcial	Definir retención y analítica.
Incidentes	Sin plan formal.	No implementado	Definir respuesta y contacto.

Fuente: Elaboración propia a partir del código y la configuración.

26.6 ISO 9001 — gestión documental y calidad

La evaluación se limita a los principios pertinentes al proyecto documental y al control de calidad del software. No existe evidencia de un sistema de gestión de la calidad certificado.

Tabla 26. Prácticas de gestión de la calidad

Práctica	Evidencia	Estado	Observación
Control documental	Manuales v3.1, README y docs.	Parcial	Aprobar versión y revisión.
Versiones	package-lock fija dependencias.	Implementado	Actualizar con revisión.

Práctica	Evidencia	Estado	Observación
Trazabilidad	Git, requisitos y pruebas.	Parcial	Vincular cambios a requisitos.
Revisión	Revisiones informales; sin acta.	Parcial	Checklist de release.
Mejora	Auditorías e inventario de pendientes.	Parcial	Seguimiento de hallazgos.
Evidencias	CI, tests, build y capturas.	Parcial	Conservar reportes por release.

Fuente: Elaboración propia.

26.7 ISO/IEC 25023 — métricas de calidad del producto

Solo se proponen métricas que pueden calcularse con evidencia del repositorio. Cuando no existen datos suficientes se indica **pendiente** y se describe la evidencia necesaria; no se inventan valores.

Tabla 27. Métricas ISO/IEC 25023:2016

Métrica	Definición operativa	Dato disponible	Estado
Cobertura de requisitos	Requisitos con evidencia / requisitos catalogados.	Catálogo de 15 requisitos; no calculado.	Pendiente
Tasa de éxito unitarias	Pruebas aprobadas / ejecutadas.	Sin ejecución actual; 90 definiciones estáticas.	Pendiente
Tasa de éxito E2E	Escenarios aprobados / ejecutados.	25/25 histórico (6AP), no actual.	Pendiente
Core Web Vitals	LCP, INP, CLS en producción.	Sin Lighthouse/APM.	Pendiente
Violaciones axe	Violaciones A/AA por página.	Axe histórico sin violaciones en E2E; no actual.	Pendiente
Tamaño del bundle	KB gzip del build.	Sin build actual.	Pendiente
Cobertura de pruebas	Líneas/branches cubiertas.	Sin reporte de cobertura.	Pendiente
Errores de cliente	Errores por sesión/página.	Sin monitoreo de producción.	Pendiente

Fuente: Elaboración propia; las métricas no sustituyen una evaluación formal.

26.8 ISO/IEC/IEEE 29119 — pruebas de software

Se documenta lo que existe: estrategia, archivos de prueba, datos, resultados históricos y evidencias. No se declaran pruebas ejecutadas sobre el commit actual si no se ejecutaron; los resultados históricos se citan como tales.

Tabla 28. Elementos de prueba ISO/IEC/IEEE 29119

Elemento	Evidencia	Estado	Limitación
Estrategia	Vitest + Testing Library; Playwright E2E; ESLint/build en CI.	Parcial	No ejecutada en el commit actual.
Casos de prueba	32 archivos unitarios; 11 specs E2E.	Parcial	No hay catálogo formal externo.
Datos de prueba	Fixtures de test; E2E con base desechable y Mailpit.	Implementado	E2E requiere entorno completo.
Resultados esperados	Aserciones y expectativas en cada test.	Implementado	No se documentan por caso.
Resultados obtenidos	90/90 y 25/25 históricos; 98/98 en v2.0.	Parcial	Cifras no reconciliadas.
Evidencias	CI.md, E2E-GATE.md, docs/entorno-pruebas-seguro.md.	Parcial	Falta reporte del commit actual.
Defectos	Inventario de pendientes y notas de riesgos.	Parcial	Sin registro formal de defectos.

Fuente: Elaboración propia a partir de CI.md, docs/entorno-pruebas-seguro.md, E2E-GATE.md y los archivos de prueba.

26.9 Matriz de trazabilidad y cumplimiento (resumen)

Tabla 29. Matriz de trazabilidad y cumplimiento — frontend web

Norma	Apartado / característica	Componente	Evidencia	Estado	Observación	Recomendación
ISO/IEC 25010	Adecuación funcional	Frontend	55 rutas/48 páginas	Parcial	Cobertura amplia; pruebas no actuales.	Validar suite y E2E.
ISO/IEC 25010	Capacidad de interacción	Frontend	ARIA, foco, axe	Parcial	Accesibilidad no reejecutada.	Auditoría WCAG.
ISO/IEC 25010	Seguridad	Frontend	Cookie, CSRF, escape	Parcial	Sin SCA/pentest.	Añadir análisis de dependencias.
ISO/IEC 25010	Mantenibilidad	Frontend	Componentes y 32 tests	Parcial	Sin cobertura.	Medir cobertura y complejidad.
ISO/IEC 12207	Verificación	Frontend	Vitest/Playwright/CI	Parcial	No actual.	Ejecutar en el commit.
ISO/IEC 29148	Requisitos	Frontend	Catálogo RF-01..RF-15	Parcial	Sin baseline.	Aprobar requisitos.
ISO/IEC 27001	Control de acceso	Frontend	ProtectedRoute + backend	Parcial	No SGSI.	Probar escenarios negativos.
ISO 9001	Control documental	Frontend	Manuales y README	Parcial	Sin aprobación formal.	Registrar revisión.
ISO/IEC 25023	Core Web Vitals	Frontend	Sin medición	Pendiente	No hay APM.	Medir LCP/INP/CLS.
ISO/IEC/IEEE 29119	Documentación de pruebas	Frontend	Vitest/Playwright	Parcial	Sin catálogo formal.	Estandarizar casos.

Fuente: Elaboración propia; la matriz completa está en Auditoria_Documentacion.

26.10 Normas complementarias

El manual v1.1 citaba normas adicionales. Se conservan y se clasifican según su aplicabilidad real, sin atribuirles conformidad ni certificación. La referencia histórica al antiguo anexo de normas se elimina porque ese anexo no existe en este documento.

Tabla 30. Normas complementarias y aplicabilidad

Norma	Aplicación en ProyecTwin	Evidencia	Estado	Justificación
ISO 14001:2026	No se atribuye un sistema de gestión ambiental al producto software.	No hay política, aspectos ambientales, objetivos ni auditoría ambiental.	No evidenciado / no aplicable	La digitalización de documentos es una práctica indirecta, no un SGA.
ISO 37001:2025	No hay sistema antisoborno ni canal ético formal.	audit_logs y bug_reports son controles de trazabilidad, no controles antisoborno.	No evidenciado / no aplicable	No existe política, evaluación de riesgos de soborno ni debida diligencia de terceros.
ISO 10005:2018	Planificación de la verificación y planes de calidad por release.	CI, suites PHPUnit/Vitest/Playwright, E2E-GATE, checklists de instalación y release.	Parcialmente evidenciado	Existen prácticas de planificación, pero no un plan de calidad formal firmado por release.
ISO/IEC 9126-1:2001	Referencia histórica del modelo de calidad.	Bibliografía y comparación con ISO/IEC 25010.	No aplicable (retirada)	Está retirada; el modelo vigente es ISO/IEC 25010:2023.
ISO/IEC 33001:2015 y familia 33000	Vocabulario para ordenar procesos; no se realizó evaluación de capacidad.	Documentación de procesos y este manual.	No evidenciado	No hay modelo de evaluación, assessor ni rating de capacidad.

Fuente: Manual v1.1 y contraste con el repositorio; consulta ISO del 04/11/2026.

26.11 Reglas de uso normativo

- Las normas se citan por edición y año; no se usan como afirmación de certificación.
- No se reproducen numerales, cláusulas ni textos protegidos; las definiciones son propias.
- Se distingue entre referencia, buena práctica, control implementado, conformidad demostrada y certificación.
- Cada recomendación se vincula con un archivo, una configuración o una prueba concreta.
- La ausencia de evidencia se registra como pendiente o no verificable, nunca como cumplimiento.
- La seguridad funcional (*safety*) no aplica al dominio académico actual; se documenta como no aplicable y se revisará si el sistema incorpora decisiones automatizadas con impacto crítico.

27. Despliegue

El build se sirve con nginx en el mismo origen que la API. El Dockerfile compila frontend/ con Node 22, copia dist/ a backend/public y nginx aplica fallback a index.html para React Router. En desarrollo se usa el proxy de Vite; en preview se usa el proxy de preview.

Tabla 31. Consideraciones de despliegue

Aspecto	Configuración	Pendiente
Build	npm ci + npm run build; VITE_API_URL=/v1.	Verificar en CI/commit actual.
Servidor	nginx sirve dist/ con fallback SPA.	Probar rutas profundas.
API	Mismo origen /v1 y /sanctum.	Validar cookies y CSRF.
Caché	Assets con hash; index sin caché prolongada.	Revisar cabeceras.
Manual/APK	public/manual y public/apk; /apk responde 404 y .dockerignore excluye el APK.	No se anuncia descarga mientras no exista URL pública autorizada.
Observabilidad	Sin APM ni RUM.	Implementar monitoreo.

Fuente: Dockerfile; docker/nginx.conf; vite.config.js.

No se verificó el despliegue real en Railway ni en un servidor nginx en esta auditoría. La evidencia es la configuración versionada.

28. Solución de problemas

Tabla 32. Diagnóstico del frontend

Síntoma	Causa probable	Solución
Pantalla en blanco	Error de render o build incompleto.	Revisar consola; probar ErrorBoundary; rebuild.
ECONNREFUSED 8000	Backend apagado.	php artisan serve.
404 en /v1	Proxy o VITE_API_URL incorrecto.	Revisar .env y vite.config.js.
419 repetido	CSRF/cookies desalineadas.	Limpiar cookies; revisar SANCTUM_STATEFUL_DOMAINS.
401 al recargar	Cookie no enviada o sesión vencida.	Revisar same-origin/credenciales; login.
403 de rol	Ruta no permitida para el rol.	Usar la ruta correcta o cuenta con permiso.
Lint falla	Nueva configuración fuera de globals.	Añadir globals/archivo a eslint.config.js.
Test E2E no arranca	Falta Mailpit/base/guarda.	Seguir docs/entorno-pruebas-seguro.md.
Node incompatible	Versión fuera del rango de Vite 8.	Usar Node 20.19+/22.12+/24+.

Fuente: frontend/README.md; CI.md; experiencia de operación.

29. Mantenimiento

- Actualizar dependencias con npm y revisar changelog; ejecutar lint, test y build.
- Mantener las rutas y roles en App.jsx y constants/routes.js como fuente única.
- Revisar la accesibilidad y las pruebas axe en cada release.
- Medir Core Web Vitals y el tamaño del bundle.
- Mantener el manual de usuario y el botón APK condicionados a una URL pública autorizada; no anunciar el APK mientras /apk responda 404.
- Revisar la caché de GET y la invalidación tras escrituras.
- No introducir secretos en VITE_*.

Deuda técnica observada: ausencia de cobertura, SCA, monitoreo RUM y pruebas de regresión visual; cifras de tests no reconciliadas; dependencia de E2E para contratos. No se modificó el código.

30. Conclusiones

La SPA se identifica en el código como una aplicación React 19/Vite 8 con experiencias separadas para aprendiz, instructor y administrador, rutas protegidas, sesión por cookie, CSRF y servicios centralizados. La revisión v4.2 corrige los conteos, las referencias de anexos, la distinción entre variables y el alcance del documento. No se ejecutaron las suites ni el build en este entorno.

Las brechas principales son de evidencia y operación: la suite no se reejecutó, no hay cobertura ni RUM, no hay SCA y las cifras de pruebas están dispersas. No se declara conformidad ni certificación ISO.

31. Anexos

31.1 Inventario completo de rutas

Tabla 33. Rutas declaradas en App.jsx

Ruta	Acceso	Componente / destino
/	Público	Home
/login	Público	Login
/recuperar-contrasena	Público	RecuperarContrasena
/restablecer-contrasena	Público	RestablecerContrasena
/activar-cuenta	Público	ActivarCuenta
/aprendiz/dashboard	Aprendiz	DashboardAprendiz
/aprendiz/propuestas	Aprendiz	Propuestas
/aprendiz/similitudes	Aprendiz	Similitudes
/aprendiz/ficha	Aprendiz	MiFicha
/aprendiz/alertas	Aprendiz	AlertasAprendiz
/aprendiz/reportar-falla	Aprendiz	ReportarFallaAprendiz
/aprendiz/perfil	Aprendiz	MiPerfil
/aprendiz/detalle-proyecto/:id	Aprendiz	DetalleProyecto
/aprendiz/detalle-similitud/:id	Aprendiz	DetalleSimilitud
/aprendiz/detalle-ficha/:id	Aprendiz	DetalleFicha
/aprendiz/perfil-companero/:id	Aprendiz/Instructor	DetalleCompanero
/aprendiz/perfil-instructor	Aprendiz	DetalleInstructor
/aprendiz/analizando-proyecto	Aprendiz	AnalizandoProyecto
/aprendiz/resultado-analisis	Aprendiz	ResultadoAnalisis
/instructor/dashboard	Instructor	DashboardInstructor
/instructor/revision-propuestas	Instructor	RevisionPropuestas
/instructor/alertas	Instructor	AlertasInstructor
/instructor/perfil	Instructor	PerfilInstructor
/instructor/detalle-proyecto/:id	Instructor	DetalleProyectoInstructor
/instructor/detalle-similitud/:id	Instructor	DetalleSimilitudInstructor
/instructor/similitudes	Instructor	SimilitudesInstructor
/instructor/reportar-falla	Instructor	ReportarFallaInstructor
/instructor/fichas	Instructor	Fichas
/instructor/detalle-ficha/:id	Instructor	DetalleFichaInstructor
/instructor/directorio-ficha/:id	Instructor	DirectorioFichaInstructor
/admin/dashboard	Admin	DashboardAdmin
/admin/usuarios	Admin	Usuarios
/admin/redes-conocimiento	Admin	RedesConocimiento
/admin/config-similitud	Admin	ConfigSimilitud
/admin/fichas	Admin	FichasAdmin
/admin/bitacora	Admin	Bitacora
/admin/detalle-ficha/:id	Admin	DetalleFichaAdmin
/admin/proyectos	Admin	ProyectosAdmin
/admin/detalle-proyecto/:id	Admin	DetalleProyectoAdmin
/admin/similitudes	Admin	SimilitudesAdmin
/admin/detalle-similitud/:id	Admin	DetalleSimilitudAdmin
/admin/detalle-usuario/:id	Admin	DetalleUsuario
/admin/reportes-fallas	Admin	ReportesFallas
/admin/detalle-reporte/:id	Admin	DetalleReporte
/admin/notificaciones	Admin	NotificacionesAdmin
/admin/perfil	Admin	PerfilAdmin
/aprendiz/mis-proyectos	Redirección	→ /aprendiz/propuestas
/aprendiz/nuevo-proyecto	Redirección	→ /aprendiz/propuestas?crear=1
/aprendiz/unirse-ficha	Redirección	→ /aprendiz/ficha
/instructor/gestionar-fichas	Redirección	→ /instructor/fichas
/instructor/crear-ficha	Redirección	→ /instructor/fichas?crear=1
/admin/gestion-usuarios	Redirección	→ /admin/usuarios
/admin/nuevo-usuario	Redirección	→ /admin/usuarios?crear=1
*	Público	PaginaNoEncontrada

Fuente: frontend/src/App.jsx (55 declaraciones: 48 páginas + 7 redirecciones).

31.2 Scripts

Tabla 34. Scripts de package.json

Script	Comando	Uso
dev	vite	Servidor de desarrollo.
build	vite build	Build de producción.
lint	eslint .	Lint completo.
test	vitest run	Pruebas unitarias.

Script	Comando	Uso
test:e2e	php ../backend/scripts/run-e2e.php	E2E con base desechable.
quality	lint + test + build	Gate local.
preview	vite preview	Servir dist/.

Fuente: frontend/package.json.

32. Glosario

Tabla 35. Glosario técnico

Término	Definición
API REST	Interfaz HTTP usada por la SPA para consumir Laravel bajo el prefijo /v1.
SPA	Single Page Application: la interfaz se carga una vez y navega en el cliente con React Router.
React	Biblioteca de interfaz declarativa usada para construir la SPA.
Vite	Herramienta de desarrollo y empaquetado; el proxy redirige /v1 y /sanctum al backend.
React Router	Librería de navegación cliente; declarada en App.jsx.
Componente	Unidad reutilizable de interfaz; los componentes compartidos viven en src/components.
Layout	Estructura visual común para un grupo de páginas.
Contexto	Estado global acotado; AuthContext, ThemeContext y MotorContext.
Hook	Función reutilizable de React; por ejemplo, useFocusTrap.
ProtectedRoute	Componente que valida sesión y rol antes de renderizar una página.
CSRF	Protección de escrituras mediante cookie XSRF-TOKEN y cabecera X-XSRF-TOKEN.
Cookie httpOnly	Cookie de sesión no legible desde JavaScript; la SPA no guarda el token en localStorage.
Sanctum	Paquete Laravel que emite la cookie de sesión y protege la API.
VITE_API_URL	Prefijo de API compilado en el bundle; por defecto /v1.
PROYECTWIN_API	URL destino del proxy de Vite; no se incluye en el bundle del navegador.
Lazy / Suspense	Carga diferida de páginas y fallback mientras se resuelve el módulo.
Core Web Vitals	Métricas de experiencia de carga e interacción; no se midieron en esta auditoría.
Accesibilidad	Uso de HTML semántico, ARIA, foco y teclado; se apoya en pruebas axe.
Vitest	Runner de pruebas unitarias de la SPA.
Playwright	Runner de pruebas end-to-end y accesibilidad.
E2E	Prueba de extremo a extremo que recorre la SPA y el backend en un entorno aislado.

Fuente: Elaboración propia con base en el código auditado.

33. Referencias

33.1 Referencias normativas

Las ediciones se contrastaron en el catálogo oficial de ISO el 4 de noviembre de 2026. No se reproduce el texto protegido de las normas; las definiciones y valoraciones son propias. La vigencia se registra solo cuando la página oficial la confirma; en caso contrario se indica como pendiente.

Tabla 36. Normas y ediciones de referencia

Norma	Edición / estado	Fuente oficial	Uso y limitación
ISO/IEC 25010	2023 (Ed. 2); publicada	https://www.iso.org/standard/78176.html	Modelo de calidad de producto; nueve características.
ISO/IEC/IEEE 12207	2026 (Ed. 2); publicada; 2017 retirada	https://www.iso.org/standard/90219.html	Procesos del ciclo de vida del software.
ISO/IEC/IEEE 29148	2018 (Ed. 2); publicada; confirmada en 2024; en revisión y con DIS 29148 en desarrollo	https://www.iso.org/standard/72089.html	Ingeniería y calidad de requisitos; no se declara conformidad.
ISO/IEC 27001	2022 + Amd 1:2024; publicada	https://www.iso.org/standard/27001	Referencia de gestión de seguridad; no certificación.
ISO 9001	2026; publicada; 2015 retirada	https://www.iso.org/standard/9001	Referencia de gestión de la calidad; no certificación.
ISO/IEC 25023	2016 (Ed. 1); vigente y en revisión; reemplazo previsto por ISO/IEC DIS 25000-23.2	https://www.iso.org/standard/35747.html	Medición de calidad de producto; sin datos actuales.
ISO/IEC/IEEE 29119-1	2022 (Ed. 2); publicada	https://www.iso.org/standard/81291.html	Conceptos generales de pruebas.

Norma	Edición / estado	Fuente oficial	Uso y limitación
ISO/IEC/IEEE 29119-2	2021 (Ed. 2); publicada	https://www.iso.org/standard/79428.html	Procesos de prueba.
ISO/IEC/IEEE 29119-3	2021 (Ed. 2); publicada	https://www.iso.org/standard/79429.html	Documentación de prueba.
ISO/IEC/IEEE 29119-4	2021 (Ed. 2); publicada	https://www.iso.org/standard/79430.html	Técnicas de prueba.
ISO/IEC/IEEE 29119-5	2024 (Ed. 2); publicada	https://www.iso.org/standard/87233.html	Pruebas dirigidas por palabras clave.
ISO 14001	2026 (Ed. 4); publicada	https://www.iso.org/standard/14001	Referencia complementaria; no aplica al producto software.
ISO 37001	2025 (Ed. 2); publicada	https://www.iso.org/standard/37001	Referencia complementaria; no hay SGAS.
ISO 10005	2018 (Ed. 3; corr. 2019); confirmada en 2023	https://www.iso.org/standard/70398.html	Guía de planes de calidad; aplicación parcial.
ISO/IEC 9126-1	2001; retirada	https://www.iso.org/standard/22749.html	Referencia histórica; sustituida por 25010.
ISO/IEC 33001	2015; confirmada en 2026	https://www.iso.org/standard/54175.html	Vocabulario de evaluación de procesos; sin evaluación formal.

Fuente: ISO.org; consulta del 04/11/2026.

Limitación: las partes ISO/IEC 33002, 33003 y 33004 no se verificaron individualmente; no se afirma su vigencia.

33.2 Referencias técnicas del frontend

Tabla 37. Documentación técnica de referencia

Tecnología	URL
React	https://react.dev/
React Router	https://reactrouter.com/
Vite	https://vite.dev/
Vitest	https://vitest.dev/
Playwright	https://playwright.dev/
Testing Library	https://testing-library.com/
ESLint	https://eslint.org/
Node.js	https://nodejs.org/docs/
npm	https://docs.npmjs.com/
MDN Web Docs	https://developer.mozilla.org/

Fuente: Sitios oficiales; consulta del 04/11/2026.

33.3 Archivos clave del frontend

Tabla 38. Mapa de archivos de referencia

Ruta	Contenido
frontend/src/App.jsx	Declaración de las 55 rutas y control de acceso.
frontend/src/pages/	48 componentes de página por rol y portal público.
frontend/src/components/	Componentes reutilizables, layouts y variantes Base.
frontend/src/contexts/	AuthContext, ThemeContext y MotorContext.
frontend/src/lib/api.js	Transporte, CSRF, caché y manejo de 401/419.
frontend/src/lib/recursos.js	Servicios de acceso a la API por recurso.
frontend/e2e/	11 especificaciones Playwright.
frontend/vite.config.js	Proxy /v1 y /sanctum; build de la SPA.
frontend/playwright.config.js	E2E, preview 4173 y guardas de base desechable.
frontend/package.json	Scripts y dependencias de la SPA.
Dockerfile	Build de la SPA y empaquetado con la API.
docker/nginx.conf	SPA, /v1, /sanctum y cabeceras de seguridad.

Fuente: Elaboración propia.